Node.js Built-in Modules

fs • path • fs/promises — Complete Cheat Sheet with Explanations

The **fs** module lets Node.js read, write, update, rename, and delete files and directories on your computer. It comes built-in — no install needed. Methods ending in **Sync** block the program until done; callback versions run asynchronously.

Import: `const fs = require('fs');`

---

1. fs.readFileSync()

Reads the ENTIRE content of a file and returns it as a string (or Buffer). Your program waits here until the file is fully read — simple but blocks execution.

Syntax: `fs.readFileSync(path, encoding)`

Code:

```
const fs = require('fs');
const data = fs.readFileSync('hello.txt', 'utf8');
console.log(data);
```

Output:

```
Hello, World!
```

■ *Throws an error if file doesn't exist. Use try/catch to handle it safely.*

---

2. fs.readFile() (async)

Reads a file WITHOUT blocking the program. Node.js continues running other code and calls your callback function when the file is ready. Preferred in web servers.

Syntax: `fs.readFile(path, encoding, callback)`

Code:

```
fs.readFile('hello.txt', 'utf8', (err, data) => {
if (err) throw err;
console.log(data);
});
```

Output:

```
Hello, World!
```

■ *The callback receives (error, data). Always check err first.*

---

3. fs.writeFileSync()

Creates a new file (or OVERWRITES an existing one) with the given content. If the file does not exist, it is created automatically.

Syntax: `fs.writeFileSync(path, data, encoding)`

Code:

```
fs.writeFileSync('output.txt', 'Learning Node.js!', 'utf8');
console.log('File written!');
```

Output:

```
File written!
```

■ *Overwrites existing content. Use fs.appendFileSync() to add instead of replace.*

---

4. fs.appendFileSync()

Adds content to the END of a file without deleting what is already there. Creates the file if it does not exist.

Syntax: `fs.appendFileSync(path, data)`

Code:

```
fs.appendFileSync('log.txt', 'New log entry\n');
console.log('Appended!');
```

Output:

```
Appended!
```

---

5. fs.existsSync()

Checks whether a file or folder EXISTS at the given path. Returns true or false — very useful before reading or deleting.

Syntax: `fs.existsSync(path) → boolean`

Code:

```
if (fs.existsSync('config.json')) {
console.log('File found!');
} else {
console.log('File missing!');
}
```

Output:

```
File found!
```

---

6. fs.renameSync()

Renames a file or MOVES it to a different location. Think of it as 'cut and paste' for files.

Syntax: `fs.renameSync(oldPath, newPath)`

Code:

```
fs.renameSync('old.txt', 'new.txt');
console.log('Renamed!');
```

Output:

```
Renamed!
```

---

7. fs.unlinkSync()

DELETES a file permanently. There is no recycle bin — the file is gone immediately.

Syntax: `fs.unlinkSync(path)`

Code:

```
fs.unlinkSync('temp.txt');
console.log('Deleted!');
```

Output:

```
Deleted!
```

■ *Throws if file doesn't exist. Check with existsSync() first.*

---

8. fs.mkdirSync()

Creates a new DIRECTORY (folder). Pass { recursive: true } to create nested folders all at once.

Syntax: `fs.mkdirSync(path, options?)`

Code:

```
fs.mkdirSync('myFolder/subFolder', { recursive: true });
console.log('Folder created!');
```

Output:

```
Folder created!
```

---

9. fs.readdirSync()

Returns an ARRAY of all file and folder names inside a directory — like running 'ls' in your terminal.

Syntax: `fs.readdirSync(path) → string[]`

Code:

```
const files = fs.readdirSync('./myFolder');
console.log(files);
```

Output:

```
[ 'index.js', 'data.json', 'README.md' ]
```

---

10. fs.statSync()

Returns METADATA about a file: size, creation date, whether it is a file or directory, and more.

Syntax: `fs.statSync(path)` → `Stats object`

Code:

```
const info = fs.statSync('hello.txt');
console.log(info.size); // size in bytes
console.log(info.isFile()); // true or false
```

Output:

```
13
true
```

The **path** module helps you build, split, and normalise file paths correctly across operating systems. On Windows paths use backslash (\\), on Mac/Linux they use forward slash (/). path handles all that for you automatically.

Import: `const path = require('path');`

---

1. path.join()

Joins multiple path segments into ONE clean path, inserting the correct separator for the OS and removing duplicate slashes.

Syntax: `path.join(...segments)` → `string`

Code:

```
const p = path.join('users', 'john', 'docs', 'file.txt');
console.log(p);
```

Output:

```
users/john/docs/file.txt
```

■ *Always use path.join instead of string concatenation — it handles OS differences.*

---

2. path.resolve()

Builds an ABSOLUTE path from the current working directory. Similar to join but always returns a full path starting from the root.

Syntax: `path.resolve(...segments)` → `string`

Code:

```
const abs = path.resolve('src', 'index.js');
console.log(abs);
```

Output:

```
/home/user/project/src/index.js
```

---

3. path.basename()

Extracts the FILE NAME from a full path — optionally removing the extension too.

Syntax: `path.basename(path, ext?)` → `string`

Code:

```
console.log(path.basename('/home/user/file.txt'));
console.log(path.basename('/home/user/file.txt', '.txt'));
```

Output:

```
file.txt
file
```

---

4. path.dirname()

Returns the DIRECTORY part of a path — everything except the final file name.

Syntax: `path.dirname(path)` → `string`

Code:

```
console.log(path.dirname('/home/user/docs/file.txt'));
```

Output:

```
/home/user/docs
```

---

5. path.extname()

Returns the FILE EXTENSION including the dot. Useful for checking what type of file something is.

Syntax: `path.extname(path)` → `string`

Code:

```
console.log(path.extname('image.png'));
console.log(path.extname('archive.tar.gz'));
```

Output:

```
.png
.gz
```

---

6. path.parse()

Breaks a path into its COMPONENTS: root, dir, base, ext, and name. Returns an object — great for inspecting paths.

Syntax: `path.parse(path)` → `object`

Code:

```
const info = path.parse('/home/user/file.txt');
console.log(info);
```

Output:

```
{ root: '/', dir: '/home/user',
base: 'file.txt', ext: '.txt', name: 'file' }
```

---

7. path.format()

Does the OPPOSITE of path.parse() — takes a path object and reassembles it into a string.

Syntax: `path.format(pathObject)` → `string`

Code:

```
const p = path.format({ dir: '/home/user', name: 'file', ext: '.txt' });
console.log(p);
```

Output:

```
/home/user/file.txt
```

---

8. path.isAbsolute()

Returns true if the path is ABSOLUTE (starts from root like / or C:\\), false if it is relative like './folder'.

Syntax: `path.isAbsolute(path)` → `boolean`

Code:

```
console.log(path.isAbsolute('/usr/bin'));
console.log(path.isAbsolute('./bin'));
```

Output:

```
true
false
```

---

9. path.relative()

Calculates the RELATIVE path from one location to another — like giving directions from point A to point B.

Syntax: `path.relative(from, to)` → `string`

Code:

```
const rel = path.relative('/home/user', '/home/user/docs/file.txt');
console.log(rel);
```

Output:

```
docs/file.txt
```

---

10. path.normalize()

Cleans up a messy path by resolving '..' and '.' segments and fixing extra slashes.

Syntax: `path.normalize(path)` → `string`

Code:

```
console.log(path.normalize('/home//user/../user/./docs'));
```

Output:

```
/home/user/docs
```

■ *Special path variables: __filename = full path of current file | __dirname = directory of current file. Use with path.join(__dirname, 'data.json').*

The **fs/promises** module is the MODERN way to work with files using **async/await** syntax. Instead of callbacks or blocking Sync methods, every function returns a Promise. Your code stays clean and non-blocking.

Import: const fs = require('fs/promises'); // or: import fs from 'fs/promises';

---

1. fs.readFile()

Reads an entire file asynchronously and returns its content as a string. Use await inside an async function — no callback needed.

Syntax: await fs.readFile(path, encoding) → string

Code:

```
const fs = require('fs/promises');
async function main() {
const data = await fs.readFile('hello.txt', 'utf8');
console.log(data);
}
main();
```

Output:

```
Hello, World!
```

■ *Wrap in try/catch to handle errors gracefully.*

---

2. fs.writeFile()

Creates or OVERWRITES a file with new content asynchronously. Waits until the write is complete before moving on.

Syntax: await fs.writeFile(path, data, encoding?)

Code:

```
async function save() {
await fs.writeFile('output.txt', 'Saved!', 'utf8');
console.log('Done writing');
}
```

---

3. fs.appendFile()

Adds content to the END of a file without removing existing data. Creates the file if it does not exist.

Syntax: await fs.appendFile(path, data)

Code:

```
async function log(msg) {
await fs.appendFile('app.log', msg + '\n');
}
log('Server started');
```

---

4. fs.mkdir()

Creates a directory asynchronously. Pass { recursive: true } to safely create nested directories without error if they already exist.

Syntax: await fs.mkdir(path, options?)

Code:

```
await fs.mkdir('logs/2024/jan', { recursive: true });
console.log('Directories created');
```

Output:

```
Directories created
```

---

5. fs.readdir()

Returns an array of file names inside a directory asynchronously. Combine with a loop to process each file.

Syntax: `await fs.readdir(path) → string[]`

Code:

```
const files = await fs.readdir('./src');
for (const file of files) {
console.log(file);
}
```

Output:

```
index.js
utils.js
config.json
```

---

6. fs.unlink()

Deletes a file asynchronously. The operation is permanent — there is no undo.

Syntax: `await fs.unlink(path)`

Code:

```
await fs.unlink('temp.txt');
console.log('File deleted');
```

Output:

```
File deleted
```

■ *Throws if the file does not exist.*

---

7. fs.rename()

Renames or MOVES a file asynchronously — works like 'mv' in the terminal.

Syntax: `await fs.rename(oldPath, newPath)`

Code:

```
await fs.rename('draft.txt', 'final.txt');
console.log('Renamed!');
```

Output:

```
Renamed!
```

---

8. fs.stat()

Returns file METADATA — size, timestamps, and whether it is a file or directory — without reading the file content.

Syntax: `await fs.stat(path) → Stats object`

Code:

```
const stats = await fs.stat('image.png');
console.log(stats.size); // bytes
console.log(stats.isFile()); // true
```

Output:

```
204800
true
```

---

9. fs.copyFile()

Copies a file from SOURCE to DESTINATION asynchronously. Overwrites destination by default; use COPYFILE_EXCL flag to prevent overwrite.

Syntax: `await fs.copyFile(src, dest, flags?)`

Code:

```
await fs.copyFile('original.txt', 'backup.txt');
console.log('Copied!');
```

Output:

```
Copied!
```

---

10. fs.rm()

Removes files AND directories asynchronously. Pass { recursive: true, force: true } to delete entire folder trees — the modern replacement for fs.rmdir().

Syntax: `await fs.rm(path, options?)`

Code:

```
await fs.rm('build', { recursive: true, force: true });
console.log('Folder removed');
```

Output:

```
Folder removed
```

■ *Introduced in Node.js 14.14. Use instead of deprecated fs.rmdir({ recursive }).*

Task	fs (sync)	fs (callback)	fs/promises
Read a file	`readFileSync()`	`readFile(cb)`	`await readFile()`
Write a file	`writeFileSync()`	`writeFile(cb)`	`await writeFile()`
Append to file	`appendFileSync()`	`appendFile(cb)`	`await appendFile()`
Delete a file	`unlinkSync()`	`unlink(cb)`	`await unlink()`
Rename / Move	`renameSync()`	`rename(cb)`	`await rename()`
Copy a file	–	`copyFile(cb)`	`await copyFile()`
Create folder	`mkdirSync()`	`mkdir(cb)`	`await mkdir()`
List folder	`readdirSync()`	`readdir(cb)`	`await readdir()`
File info / stat	`statSync()`	`stat(cb)`	`await stat()`
Check existence	`existsSync()`	`access(cb)`	`await access()`
Remove folder	`rmSync()`	`rm(cb)`	`await rm()`

Tip: Use fs/promises + async/await for all new code. Use Sync methods only in startup scripts or CLI tools where blocking is acceptable. Combine path.join(__dirname, filename) with any fs call to safely build file paths.